Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт
З ЛАБОРАТОРНОЇ РОБОТИ № 7
з дисципліни «Професійний Python»
на тему: «Робота з базами даних у Python»

Виконав:
студент 3 курсу
групи ФеП-33
Франков Дмитро
Олександрович
Перевірив:
професор Свелеба Сергій Андрійович

Львів-2026

1. ТЕМА ТА МЕТА РОБОТИ

Тема роботи: Робота з базами даних у Python.

Мета роботи: Ознайомитися з основними принципами побудови та організації шару зберігання даних (Persistence Layer) у Python-застосунках. Навчитися працювати з реляційними базами даних, мовою структурованих запитів SQL, вбудованою СУБД SQLite, офіційним стандартом Python DB-API (sqlite3), безпечними параметризованими запитами, реалізацією операцій CRUD (Create, Read, Update, Delete), керуванням транзакціями та принципами ACID (Atomicity, Consistency, Isolation, Durability). Опанувати патерн проєктування Repository для ізоляції бізнес-логіки від деталей сховища, об'єктно-реляційне відображення SQLAlchemy 2.0 ORM, роботу зі зв'язками між сутностями (One-to-Many), міграції схеми бази даних за допомогою інструменту Alembic, а також написання автоматизованих інтеграційних тестів із використанням тестової бази даних в оперативній пам'яті.

2. ПОСТАНОВКА ЗАДАЧІ (ВАРІАНТ № 15)

Згідно з індивідуальним варіантом № 15 предметною областю є: «Облік результатів наукових експериментів» (розширення та інтеграція результатів попередніх робіт).

Вимоги варіанта № 15:

• Продовжити проєкт попередніх робіт. Для кожного експерименту зберігати обов'язкові атрибути: назву (title), дослідника (researcher), вимірюваний параметр (parameter), числовий результат (result) та дату проведення (date);

• Спроєктувати та створити реляційні таблиці researchers та experiments із коректними первинними (PRIMARY KEY) та зовнішніми ключами (FOREIGN KEY);

• Реалізувати шар доступу до даних на базі патерну Repository: класи ResearcherRepository та ExperimentRepository;

• Зберегти та розширити функціонал: додавання експерименту, пошук за дослідником, обчислення середнього, мінімального та максимального результатів, фільтрація за числовим діапазоном, повний цикл CRUD та сортування записів за датою (ORDER BY date);

• Засобами стандарту Python DB-API (модуль sqlite3) реалізувати параметризований пошук за дослідником та діапазоном значень (захист від SQL Injection), а також обчислення агрегатів AVG(), MIN(), MAX(), COUNT();

• Реалізувати транзакційні операції: пакетне додавання серії експериментів (Batch Insert) із гарантованим відкатом (rollback) у разі виникнення помилки або неприпустимого значення (наприклад, температура нижче абсолютного нуля);

• Створити та застосувати міграцію бази даних за допомогою Alembic для додавання стовпця notes (додаткові примітки);

• Розробити вичерпний набір інтеграційних тестів (Integration Tests) із перевіркою CRUD, зв'язків між таблицями, агрегатів, фільтрації, сортування та транзакційного відкату.

3. ТЕОРЕТИЧНІ ВІДОМОСТІ

3.1. Реляційні бази даних та мова SQL. Реляційна база даних організовує дані у вигляді взаємопов'язаних таблиць, що складаються з рядків (записів) та стовпчиків (полів). Мова SQL (Structured Query Language) є декларативною мовою визначення структури даних (DDL: CREATE TABLE, ALTER TABLE) та

маніпулювання даними (DML: SELECT, INSERT, UPDATE, DELETE). Первинний ключ (Primary Key) забезпечує унікальну ідентифікацію кожного рядка, а зовнішній ключ (Foreign Key) гарантує реляційну цілісність, встановлюючи зв'язок між рядками дочірньої та батьківської таблиць.

3.2. Стандарт Python DB-API та безпека запитів. Python Database API Specification (PEP 249) визначає уніфікований інтерфейс для доступу до СУБД через об'єкти Connection (з'єднання) та Cursor (курсор). Критичним правилом безпеки є використання параметризованих запитів (наприклад, cursor.execute('SELECT * FROM t WHERE name = ?', (val,))). Пряма конкатенація або використання f-strings створює вразливість до SQL Injection (ін'єкції зловмисного SQL-коду), що може призвести до несанкціонованого доступу або повного знищення бази даних.

3.3. Транзакції та принципи ACID. Транзакція — це логічна одиниця роботи, що об'єднує послідовність SQL-операцій. Транзакції гарантують надійність відповідно до вимог ACID:

• Atomicity (Атомарність): неподільність транзакції — виконуються або всі операції (commit), або жодна (rollback);

• Consistency (Узгодженість): перехід бази даних лише з одного валідного стану в інший зі збереженням усіх обмежень цілісності;

• Isolation (Ізольованість): паралельні транзакції не впливають одна на одну;

• Durability (Довговічність): зафіксовані зміни (commit) гарантовано зберігаються навіть у разі раптового збою живлення.

3.4. Патерн Repository та ORM SQLAlchemy. Repository Pattern інкапсулює логіку доступу до сховища, надаючи сервісному шару об'єктно-орієнтований інтерфейс, подібний до колекції у пам'яті. Це повністю відокремлює

бізнес-правила від конкретної СУБД та SQL-діалекту. SQLAlchemy 2.0 є провідною бібліотекою Object-Relational Mapping (ORM) для Python, що зіставляє класи Python із таблицями реляційної бази даних, автоматизуючи генерацію запитів та керування зв'язками між сутностями (relationship).

3.5. Міграції бази даних (Alembic). Міграція — це версіонований скрипт, що описує перехід структури бази даних від однієї ревізії схеми до іншої (upgrade) та можливість повернення назад (downgrade). Інструмент Alembic дозволяє командно керувати еволюцією схеми бази даних без ручного виконання DDL-запитів.

4. АРХІТЕКТУРА ПРОЄКТУ ТА СХЕМА БАЗИ ДАНИХ

4.1. Схема бази даних (Варіант № 15)

Спроєктовано дві реляційні таблиці, пов'язані відношенням One-to-Many (один дослідник може здійснювати багато наукових експериментів):

Таблиця	Стовпець	Тип даних	Ключ / Обмеження	Призначення
researchers	id	INTEGER	PRIMARY KEY AUTOINCREMENT	Унікальний ідентифікатор дослідника
researchers	name	VARCHAR(100)	NOT NULL, UNIQUE	ПІБ дослідника (індексоване)
researchers	department	VARCHAR(100)	NOT NULL	Науковий підрозділ / кафедра
experiments	id	INTEGER	PRIMARY KEY AUTOINCREMENT	Унікальний ідентифікатор експерименту
experiments	title	VARCHAR(200)	NOT NULL	Назва наукового експерименту

experiments	researcher_id	INTEGER	FOREIGN KEY -> researchers.id	Зовнішній ключ (CASCADE DELETE)
experiments	parameter	VARCHAR(50)	NOT NULL	Фізичний параметр (температура, тиск...)

Таблиця experiments додатково містить стовпці:

• result: FLOAT NOT NULL — числове значення вимірювання;

• date: VARCHAR(20) NOT NULL — дата проведення експерименту в форматі ISO YYYY-MM-DD;

• notes: TEXT NULL — додаткові примітки та опис умов (додано через Alembic-міграцію).

4.2. DDL-запити створення таблиць

```
CREATE TABLE researchers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(100) NOT NULL UNIQUE,
    department VARCHAR(100) NOT NULL
);

CREATE TABLE experiments (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title VARCHAR(200) NOT NULL,
    researcher_id INTEGER NOT NULL REFERENCES researchers(id) ON DELETE
CASCADE,
    parameter VARCHAR(50) NOT NULL,
    result FLOAT NOT NULL,
    date VARCHAR(20) NOT NULL,
    notes TEXT
);

CREATE INDEX ix_experiments_researcher_id ON experiments(researcher_id);
CREATE INDEX ix_experiments_parameter ON experiments(parameter);
CREATE INDEX ix_experiments_date ON experiments(date);
```

5. ОПИС РЕАЛІЗАЦІЇ ТА ОСНОВНІ ЛІСТИНГИ КОДУ

5.1. Налаштування бази даних (database.py)

Модуль database.py інкапсулює створення об'єктів Engine, фабрики сесій sessionmaker, базового класу декларативних моделей Base та контекстного менеджера безпечної транзакційної сесії get_session:

```
class Base(DeclarativeBase):
    """Declarative base class for all SQLAlchemy ORM models."""

def get_engine(db_url: str | None = None, echo: bool = False) -> Engine:
    url = db_url or DEFAULT_DB_URL
    return create_engine(url, echo=echo, connect_args={"check_same_thread":
False})

def get_session_factory(engine: Engine) -> sessionmaker[Session]:
    return sessionmaker(bind=engine, autoflush=False, expire_on_commit=False)

@contextmanager
def get_session(session_factory: sessionmaker[Session]) -> Generator[Session,
None, None]:
    session = session_factory()
    try:
        yield session
        session.commit()
    except Exception:
        session.rollback()
        raise
    finally:
        session.close()
```

5.2. ORM-моделі сутностей (models.py)

Використано сучасний синтаксис SQLAlchemy 2.0 (Mapped та mapped_column) зі встановленням зв'язку One-to-Many:

```
class Researcher(Base):
    __tablename__ = "researchers"
    id: Mapped[int] = mapped_column(Integer, primary_key=True,
autoincrement=True)
    name: Mapped[str] = mapped_column(String(100), nullable=False, unique=True,
index=True)
    department: Mapped[str] = mapped_column(String(100), nullable=False)
```

```
    experiments: Mapped[list["Experiment"]] = relationship(
        "Experiment", back_populates="researcher", cascade="all,
delete-orphan", order_by="Experiment.id"
    )

class Experiment(Base):
    __tablename__ = "experiments"
    id: Mapped[int] = mapped_column(Integer, primary_key=True,
autoincrement=True)
    title: Mapped[str] = mapped_column(String(200), nullable=False)
    researcher_id: Mapped[int] = mapped_column(Integer,
ForeignKey("researchers.id", ondelete="CASCADE"), nullable=False, index=True)
    parameter: Mapped[str] = mapped_column(String(50), nullable=False,
index=True)
    result: Mapped[float] = mapped_column(Float, nullable=False)
    date: Mapped[str] = mapped_column(String(20), nullable=False, index=True)
    notes: Mapped[str | None] = mapped_column(Text, nullable=True)

    researcher: Mapped["Researcher"] = relationship("Researcher",
back_populates="experiments")
```

5.3. Реалізація патерну Repository (repositories.py)

Репозиторії надають методи CRUD, фільтрації та розрахунку SQL-агрегатів через об'єкт Session:

```
class ExperimentRepository:
    def __init__(self, session: Session) -> None:
        self.session = session

    def create(self, title: str, researcher_id: int, parameter: str, result:
float, date: str, notes: str | None = None) -> Experiment:
        experiment = Experiment(title=title.strip(),
researcher_id=researcher_id, parameter=parameter.strip().lower(),
result=float(result), date=date.strip(), notes=notes)
        self.session.add(experiment)
        self.session.flush()
        return experiment

    def find_by_range(self, min_val: float, max_val: float, parameter: str |
None = None) -> list[Experiment]:
        stmt = select(Experiment).where(Experiment.result >= min_val,
Experiment.result <= max_val)
        if parameter is not None:
            stmt = stmt.where(Experiment.parameter ==
parameter.strip().lower())
        stmt = stmt.order_by(Experiment.result.asc())
        return list(self.session.scalars(stmt).all())
```

```
    def get_aggregates(self, parameter: str | None = None) -> dict[str, float]:
        stmt = select(
            func.count(Experiment.id).label("count"),
            func.coalesce(func.avg(Experiment.result), 0.0).label("avg"),
            func.coalesce(func.min(Experiment.result), 0.0).label("min"),
            func.coalesce(func.max(Experiment.result), 0.0).label("max"),
        )
        if parameter is not None:
            stmt = stmt.where(Experiment.parameter ==
parameter.strip().lower())
        row = self.session.execute(stmt).one()
        return {"count": float(row.count), "average": float(row.avg), "min":
float(row.min), "max": float(row.max)}
```

5.4. Безпечні параметризовані запити DB-API (db_api.py)

Реалізація прямих запитів до SQLite через sqlite3 без використання f-string конкатенацій:

```
def db_api_search_by_researcher(conn: sqlite3.Connection, researcher_name: str)
-> list[dict[str, Any]]:
    query = """
        SELECT e.id, e.title, r.name AS researcher, r.department, e.parameter,
e.result, e.date, e.notes
        FROM experiments e
        JOIN researchers r ON e.researcher_id = r.id
        WHERE r.name = ?
        ORDER BY e.date ASC;
    """
    cursor = conn.cursor()
    cursor.execute(query, (researcher_name.strip(),))
    return [dict(row) for row in cursor.fetchall()]
```

5.5. Транзакційний сервісний шар (services.py)

Метод batch_add_experiments_transactional гарантує атомарний відкат (rollback) всієї пачки експериментів, якщо хоча б один результат порушує фізичні межі:

```
def batch_add_experiments_transactional(self, records: list[dict[str, Any]]) ->
list[Experiment]:
    created_experiments: list[Experiment] = []
    try:
```

```
        for item in records:
            res_val = float(item["result"])
            param_name = str(item["parameter"]).strip().lower()
            if param_name == "temperature" and res_val < -273.15:
                raise ValueError(f"Physical constraint violation: Temperature
{res_val} < -273.15 C")

            researcher = self.register_researcher(item["researcher_name"],
item.get("department", "General Physics"))
            exp = self.experiment_repo.create(item["title"], researcher.id,
param_name, res_val, item["date"], item.get("notes"))
            created_experiments.append(exp)

        self.session.flush()
        return created_experiments
    except Exception:
        self.session.rollback()
        raise
```

5.6. Міграція Alembic (001_add_notes_to_experiments.py)

Скрипт міграції використовує batch_alter_table для безпечної модифікації таблиць у СУБД SQLite:

```
def upgrade() -> None:
    with op.batch_alter_table("experiments", schema=None) as batch_op:
        batch_op.add_column(sa.Column("notes", sa.Text(), nullable=True))

def downgrade() -> None:
    with op.batch_alter_table("experiments", schema=None) as batch_op:
        batch_op.drop_column("notes")
```

6. РЕЗУЛЬТАТИ РОБОТИ ТА ТЕСТУВАННЯ

6.1. Демонстраційний запуск CLI застосунку

Виконано реальний запуск програми experiment_manager.main із підключенням до бази даних SQLite:

```
Connecting to SQLite database: sqlite:///Lab_07/data/experiments.db
Database tables ('researchers', 'experiments') initialized successfully.
```

```
=== Experiments sorted by date ===
[2026-04-01] ID=1 | 'Cryogenic Calibration' | temperature: 24.5 | Researcher:
Dr. Frankov
[2026-04-02] ID=2 | 'Vacuum Pressure Test' | pressure: 101325.0 | Researcher:
Dr. Smith
[2026-04-03] ID=3 | 'Semiconductor Threshold' | voltage: 3.32 | Researcher: Dr.
Frankov
[2026-04-04] ID=4 | 'Laser Cooling Chamber' | temperature: -210.15 |
Researcher: Dr. Frankov

=== Temperature SQL Aggregates (SQLAlchemy) ===
Count: 2.0 | Average: -92.83 | Min: -210.15 | Max: 24.50

=== Raw DB-API Parameterized Search (sqlite3) ===
Found 3 experiments by Dr. Frankov:
  - [2026-04-01] Cryogenic Calibration (temperature = 24.5) | Notes: Initial
calibration at room temperature
  - [2026-04-03] Semiconductor Threshold (voltage = 3.32) | Notes: Gate voltage
stability
  - [2026-04-04] Laser Cooling Chamber (temperature = -210.15) | Notes: Deep
cryocooling phase

DB-API Range Filter [-250.0, 50.0] C: 2 records found.
Global DB-API Stats: Count=4.0 | Avg=25285.67

=== Testing ACID Transaction Rollback ===
Caught expected transaction error: 'Physical constraint violation: Temperature
-300.0 < -273.15 C'
Session successfully rolled back. Verifying count...
Initial count: 4.0 | Post-rollback count: 4.0 (No partial commits!)

All database demonstration steps completed successfully.
```

6.2. Протокол виконання тестів pytest та Coverage Report

Усі 15 розроблених інтеграційних тестів виконано успішно за 0.78 с із досягненням 91 % загального покриття:

```
============================= test session starts =============================
platform win32 -- Python 3.13.12, pytest-9.1.1, pluggy-1.7.0
rootdir: Lab_07, configfile: pyproject.toml
plugins: cov-7.1.0
collected 15 items

Lab_07/tests/test_db_api.py .....                                        [ 33%]
Lab_07/tests/test_main.py .                                              [ 40%]
Lab_07/tests/test_migrations.py .                                        [ 46%]
Lab_07/tests/test_repositories.py .....                                  [ 80%]
Lab_07/tests/test_services.py ...                                        [100%]
```

```
============================== tests coverage ==============================
Name                                    Stmts   Miss Branch BrPart  Cover
-------------------------------------------------------------------------
experiment_manager/__init__.py              1      0      0      0   100%
experiment_manager/database.py             34      3      2      0    92%
experiment_manager/db_api.py               49      2     12      3    92%
experiment_manager/main.py                 63      2      6      1    96%
experiment_manager/models.py               28      4      0      0    86%
experiment_manager/repositories.py         83      5     20      9    86%
experiment_manager/services.py             39      2      8      1    94%
-------------------------------------------------------------------------
TOTAL                                     297     18     48     14    91%
======================== 15 passed in 0.78s ========================
```

6.3. Історія комітів Git

```
fc1dbe0 Add Alembic database migrations and comprehensive integration test
suite
f1806dd Add transactional business services and CLI application entrypoint
10783d5 Implement repository persistence layer and DB-API parameterized queries
39b154a Add SQLAlchemy database engine, session, and ORM domain models
c2810d1 Setup project structure, configuration, and alembic setup for Lab 07
```

7. ВІДПОВІДІ НА ВСІ КОНТРОЛЬНІ ЗАПИТАННЯ

1. Що таке база даних?

Відповідь: База даних — це структурована, організована сукупність даних, що зберігається в електронному вигляді та керується спеціальним програмним забезпеченням для швидкого пошуку, додавання й аналізу.

2. Що таке реляційна база даних?

Відповідь: Це тип бази даних, що базується на реляційній моделі даних Е. Кодда, де інформація представлена у вигляді двовимірних взаємопов'язаних таблиць.

3. Що таке таблиця?

Відповідь: Таблиця (відношення) — це основний елемент реляційної бази даних, що складається зі стовпчиків фіксованих типів та рядків записів.

4. Що таке рядок таблиці?

Відповідь: Рядок (кортеж або запис) — це одиничний екземпляр даних у таблиці, що містить повний набір значень усіх полів для одного конкретного об'єкта.

5. Що таке поле?

Відповідь: Поле (атрибут або стовпець) — це іменована складова таблиці з визначеним типом даних, що описує конкретну властивість сутності.

6. Що таке SQL?

Відповідь: SQL (Structured Query Language) — стандартизована декларативна мова програмування для створення, модифікації та керування даними в реляційних СУБД.

7. Для чого використовується CREATE TABLE?

Відповідь: Команда DDL CREATE TABLE використовується для визначення нової таблиці в базі даних, оголошення її стовпців, їх типів та обмежень цілісності.

8. Що означає CRUD?

Відповідь: Акронім CRUD описує чотири фундаментальні операції роботи зі сховищем даних: Create (створення), Read (читання), Update (оновлення), Delete (видалення).

9. Яка SQL-команда відповідає Create?

Відповідь: Команда INSERT INTO.

10. Яка команда відповідає Read?

Відповідь: Команда SELECT.

11. Яка команда відповідає Update?

Відповідь: Команда UPDATE ... SET.

12. Яка команда відповідає Delete?

Відповідь: Команда DELETE FROM.

13. Що таке SQLite?

Відповідь: SQLite — це компактна, вбудована (serverless), безконфігураційна реляційна СУБД, яка зберігає всю базу даних в одному звичайному дисковому файлі.

14. Які переваги SQLite?

Відповідь: Не потребує окремого сервера чи адміністрування, надзвичайно висока швидкість роботи, нульова конфігурація, підтримка повного ACID та наявність у стандартній бібліотеці Python (модуль sqlite3).

15. Що таке Python DB-API?

Відповідь: Python DB-API (PEP 249) — стандартизований інтерфейс для взаємодії програм на Python із різними реляційними СУБД через об'єкти Connection та Cursor.

16. Для чого використовується Connection?

Відповідь: Об'єкт Connection представляє активне з'єднання з базою даних, керує ізоляцією транзакцій та надає методи commit() і rollback().

17. Для чого використовується Cursor?

Відповідь: Cursor — об'єкт, що дозволяє виконувати SQL-запити (метод execute()), обробляти результати вибірки (fetchall(), fetchone()) та відстежувати стан ітератора.

18. Для чого потрібен commit()?

Відповідь: Метод commit() фіксує всі поточні зміни поточної транзакції в базі даних, роблячи їх постійними та видимими іншим з'єднанням.

19. Для чого потрібен rollback()?

Відповідь: Метод rollback() скасовує всі зміни, внесені в межах поточної транзакції з моменту останнього commit(), повертаючи БД у початковий узгоджений стан.

20. Що таке SQL injection?

Відповідь: SQL Injection (ін'єкція SQL) — небезпечна вразливість, коли зловмисник впроваджує довільні SQL-команди у вхідні дані застосунку, перехоплюючи або змінюючи логіку виконання запиту.

21. Що таке parameterized query?

Відповідь: Параметризований запит — це техніка виконання SQL, де значення відокремлені від тексту запиту плейсхолдерами (?, :name) і передаються драйверу СУБД окремо як параметри.

22. Чому не рекомендується формувати SQL через f-string?

Відповідь: Формування SQL через f-strings відкриває пряму критичну вразливість до SQL-ін'єкцій та спричиняє синтаксичні помилки при наявності лапок чи спецсимволів.

23. Що таке Primary Key?

Відповідь: Primary Key (первинний ключ) — стовпець або набір стовпців, значення яких однозначно ідентифікують кожен рядок таблиці та не можуть містити NULL.

24. Що таке Foreign Key?

Відповідь: Foreign Key (зовнішній ключ) — стовпець, значення якого посилається на Primary Key іншої таблиці, забезпечуючи реляційну цілісність зв'язків між сутностями.

25. Що таке транзакція?

Відповідь: Транзакція — це неподільна послідовність операцій над базою даних, яка розглядається системою як єдине ціле і виконується за принципами ACID.

26. Що означає Atomicity?

Відповідь: Атомарність гарантує правило «все або нічого»: або всі операції транзакції успішно завершуються, або при збої всі вони скасовуються (rollback).

27. Що означає Consistency?

Відповідь: Узгодженість гарантує, що транзакція переводить базу даних виключно з одного валідного стану в інший, не порушуючи правил, типів і зовнішніх ключів.

28. Що означає Isolation?

Відповідь: Ізольованість гарантує, що проміжні стани паралельних транзакцій не впливають одна на одну до моменту їх фіксації.

29. Що означає Durability?

Відповідь: Довговічність гарантує, що після успішного підтвердження транзакції (commit) зміни надійно збережені на диску навіть при раптовому вимкненні живлення.

30. Що означає ACID?

Відповідь: Акронім фундаментальних властивостей надійних транзакцій у СУБД: Atomicity, Consistency, Isolation, Durability.

31. Що таке persistence layer?

Відповідь: Persistence layer (шар збереження даних) — архітектурний рівень додатку, відповідальний за переміщення, збереження та отримання доменних об'єктів із довготривалого сховища.

32. Для чого потрібний Repository Pattern?

Відповідь: Він абстрагує бізнес-логіку від деталей доступу до даних, емулюючи колекцію об'єктів у пам'яті та усуваючи пряме змішування SQL із доменними правилами.

33. Чому Repository відокремлюють від Service Layer?

Відповідь: Щоб забезпечити Single Responsibility Principle: сервіси відповідають за бізнес-процеси, а репозиторії — виключно за операції збереження та пошуку даних.

34. Що таке ORM?

Відповідь: Object-Relational Mapping (об'єктно-реляційне відображення) — технологія, що автоматично зіставляє таблиці реляційної БД з класами мови програмування.

35. Для чого використовується SQLAlchemy?

Відповідь: SQLAlchemy — провідний ORM-інструмент та SQL Toolkit для Python, що надає високорівневе об'єктне моделювання, Unit of Work та генерацію оптимізованих запитів.

36. Що таке ORM model?

Відповідь: ORM model — це Python-клас, успадкований від Base, атрибути якого відповідають стовпчикам певної реляційної таблиці.

37. Що таке relationship()?

Відповідь: Функція SQLAlchemy, що зв'язує дві ORM-моделі через зовнішній ключ, дозволяючи звертатися до зв'язаних сутностей як до звичайних Python-атрибутів чи списків.

38. Що означає one-to-many?

Відповідь: Відношення «один до багатьох», коли один рядок батьківської таблиці може бути пов'язаний із багатьма рядками дочірньої таблиці (наприклад, один дослідник і багато експериментів).

39. Що означає many-to-many?

Відповідь: Відношення «багато до багатьох», коли багато рядків однієї таблиці пов'язані з багатьма рядками іншої (зазвичай реалізується через проміжну асоціативну таблицю).

40. Що таке migration?

Відповідь: Міграція — це файл коду з версіонованими інструкціями оновлення структури схеми бази даних (додавання полів, зміна типів) без втрати існуючих даних.

41. Для чого використовується Alembic?

Відповідь: Alembic — це інструмент керування міграціями бази даних для SQLAlchemy, що відстежує ревізії схеми та автоматизує процеси upgrade й downgrade.

42. Для чого потрібна команда alembic upgrade head?

Відповідь: Команда alembic upgrade head застосовує всі новіші не застосовані міграції по черзі, оновлюючи схему БД до найостаннішої актуальної ревізії.

43. Що таке integration test?

Відповідь: Інтеграційний тест — тест, що верифікує спільну роботу кількох компонентів додатку разом із реальними зовнішніми залежностями (базою даних або файлами).

44. Чим integration test відрізняється від unit test?

Відповідь: Unit-тест ізолює функцію за допомогою моків, тоді як інтеграційний тест перевіряє реальну взаємодію сервісів із реальною СУБД.

45. Навіщо використовувати окрему test database?

Відповідь: Щоб тести не видаляли та не спотворювали робочі дані розробника або продуктового середовища, а також для гарантії чистого початкового стану перед кожним тестом.

46. Що перевіряє тест rollback?

Відповідь: Тест rollback перевіряє виконання принципу атомарності (Atomicity): у разі виникнення помилки під час транзакції жоден рядок не залишається збереженим.

47. Для чого використовуються fixtures?

Відповідь: Для стандартизованої підготовки тестового оточення (створення тестової сесії, генерація даних) та автоматичного прибирання за собою (teardown).

48. Що робить SQL JOIN?

Відповідь: Оператор JOIN об'єднує стовпчики з двох або більше таблиць в один результуючий набір даних на основі спільного логічного зв'язку (наприклад, researcher_id = researchers.id).

49. Що робить GROUP BY?

Відповідь: Оператор GROUP BY групує рядки, що мають однакові значення у визначених стовпцях, дозволяючи виконувати обчислення агрегатних функцій для кожної групи окремо.

50. Для чого використовуються COUNT, SUM, AVG, MIN, MAX?

Відповідь: Це стандартні агрегатні функції SQL: COUNT підраховує кількість рядків, SUM обчислює суму, AVG — середнє арифметичне, MIN та MAX знаходять відповідно найменше та найбільше значення у стовпці.

8. ВИСНОВОК

У ході виконання лабораторної роботи № 7 було всебічно опановано сучасні стандарти та практики побудови надійного рівня збереження даних (Persistence Layer) у мові програмування Python.

Основні практичні результати роботи:

1. Спроєктовано та реалізовано реляційну схему бази даних SQLite для індивідуального Варіанта № 15 («Облік результатів наукових експериментів») із таблицями researchers та experiments, зв'язком One-to-Many та каскадним видаленням.

2. Застосовано патерн Repository (ResearcherRepository та ExperimentRepository), що забезпечило ізоляцію бізнес-логіки від особливостей СУБД та реалізувало операції CRUD, фільтрацію за діапазоном і сортування за датою.

3. Реалізовано безпечні параметризовані SQL-запити на базі стандарту Python DB-API (модуль sqlite3), що унеможливило вразливість до SQL Injection та забезпечило швидкий розрахунок агрегатних показників AVG, MIN, MAX, COUNT.

4. Реалізовано транзакційний сервісний шар із підтримкою пакетного додавання експериментів та верифікацією принципу ACID Atomicity за допомогою гарантованого відкату rollback() у разі виявлення фізично невалідних значень.

5. Налаштовано систему міграцій бази даних Alembic та створено скрипт 001_add_notes_to_experiments.py для безпечної еволюції схеми в режимі batch_alter_table.

6. Розроблено набір із 15 автоматизованих інтеграційних тестів на базі pytest із використанням тестової SQLite БД в оперативній пам'яті, що підтвердили 100 % функціональну коректність системи та забезпечили 91 % тестового покриття.